

SatQuery AI (ISRO — SIH26167)

Team Roles, Responsibilities & GitHub Integration Playbook

Team Size: 6 Members

Team Leader: Peter

Project: SatQuery AI — Agentic Multimodal Remote Sensing Intelligence Platform

Target Organization: Indian Space Research Organisation (ISRO) / Space Applications Centre (SAC)

Hackathon: Smart India Hackathon (SIH) 2026 | Software Track (SIH26167)

1. Executive Team Charter & Leadership Strategy

To win the Smart India Hackathon under ISRO, our 6-person team operates on **modular engineering tracks**, **strict GitHub contracts**, and **zero-blocker mock interfaces**.

Team Leader Role (Peter): Chief Architect, Agentic Orchestrator & Integration Lead

[!TIP] **Why this leadership structure is unbeatable:**

Rather than being an abstract manager or a purely floating helper, Peter owns the **Agentic Task Orchestrator** (**AgenticTaskRouter**) itself. The orchestrator is the literal mathematical and logical spine of the entire system—it accepts queries from Vinayak's frontend, validates geospatial metadata from Misha's pipeline, routes inputs to Chhavi's AI engines, enforces parameters within Achintya's backend schemas, and verifies accuracy against Pradipti's benchmark criteria.

As **Integration Lead**, Peter acts as the **Git Master**, managing pull requests, unblocking teammates, resolving cross-module conflicts, and guaranteeing that when all 6 members push their branches, the repository compiles cleanly into a single, cohesive, award-winning platform.

SatQuery AI – 6-Person Core Team Roster	
Peter (Team Leader & Architect) - Central Agentic Orchestrator - Git Repo & Integration Lead - Cross-Module Unblocker & Guide	Pradipti (Research, QA & Pitch) - ISRO Sensor Physics & Datasets - Metric Benchmarks (mIoU/BLEU/F1) - Official 8-Slide PPT & Jury Pitch
Chhavi (AI & Deep Learning Lead) - RS-VLM & SAM-2 Specialist - ChangeFormer & Optical-SAR - MLOps: Quantization (ONNX) - GPU VRAM Optimization & Latency	Vinayak (Frontend & Secondary BE) - React + MapLibre GL Canvas - Dual-Pane Swipe Comparison Slider - Dynamic Vector Layer Overlays - Secondary Backend REST Routes
Achintya (Backend Lead & System) - FastAPI Gateway & REST Schemas - System Integration & Security - Auditable JSON Trace Logger - PDF Dossier Exporter Service	Misha (Database & Geospatial Lead) - PostGIS Spatial Database Lake - Rasterio GeoTIFF Parser & COGs - Affine Matrix Coordinate Mapping - Band Math (NDVI/NDWI/SAR dB)

2. Detailed Member Profiles, Deliverables & Code Ownership

Peter — Team Leader, Chief Architect & Agentic Orchestrator

- Domain Focus:** System Architecture, Agentic Task Orchestration, Technical Project Management, Git Integration.
- Core Responsibilities:**
- Central Agentic Orchestrator:** Implements `AgenticTaskRouter` classifying queries into Single VQA, Captioning, Region Grounding, Bi-Temporal Change, or Cross-Modal Fusion.
- Dynamic Pipeline Assembly:** Sequences preprocessing, neural backbones, affine projection, and output synthesis.
- Parameter Bounding & Safety Guardrails:** Enforces mathematical guardrails on confidence thresholds, change thresholds, and filter kernels.
- Git Master & Continuous Integration:** Manages repository branching, code reviews, PR approvals, and cross-team unblocking.
- Live Presentation Conductor:** Introduces the project architecture and coordinates live transitions during the jury round.
- Direct Code Ownership:**
- `satquery/services/orchestrator.py`
- `satquery/core/config.py`
- Root configuration, CI/CD smoke tests, and PR merges.
- Primary Tech Stack:** Python 3.11, Pydantic v2, Git, GitHub Actions.

Chhavi — AI & Deep Learning Lead + MLOps

- Domain Focus:** Neural Architectures, Vision-Language Adaptation, Segmentation, Model Quantization, GPU VRAM.
- Core Responsibilities:**

- **Tool 1 ([RS-VQA-Engine](#))**: Adapts Remote-Sensing VLM (GeoChat / RemoteCLIP / LLaVA-Earth) with LoRA weights for captioning and visual QA.
 - **Tool 2 ([SpatialGrounding-Engine](#))**: Connects Grounding DINO text grounding with SAM-2 (Segment Anything Model 2) zero-shot polygon mask generation.
 - **Tool 3 ([BiTemporalChange-Engine](#))**: Implements [ChangeFormer-V2](#) / BIT cross-attention difference heads; writes directional change reasoning (growth/decay).
 - **Tool 4 ([OpticalSARFusion-Engine](#))**: Constructs dual-stream Siamese cross-modal feature fusion for cloud-penetrating vision.
 - **MLOps & Inference Optimization**: Quantizes models using ONNX Runtime / FP16 to achieve $<2.5\text{s}$ response times; manages GPU memory pooling to eliminate CUDA Out-Of-Memory (OOM) crashes.
 - **Direct Code Ownership**:
 - [satquery/tools/vqa_engine.py](#)
 - [satquery/tools/spatial_grounding.py](#)
 - [satquery/tools/change_engine.py](#)
 - [satquery/tools/fusion_engine.py](#)
 - [satquery/mlops/](#) (model quantization and GPU optimization scripts).
 - **Primary Tech Stack**: PyTorch, Hugging Face Transformers, PEFT/LoRA, SAM-2, Grounding DINO, ONNX Runtime.
-

Pradipti — Research, Benchmark, QA & Pitch Lead

- **Domain Focus**: Remote Sensing Physics, Benchmark Validation, End-to-End Quality Assurance, Live Pitch Leadership.
 - **Core Responsibilities**:
 - **Benchmark Data Curation**: Prepares and validates official test splits from [BigEarthNet.txt](#) (BigEarthNet-MM), [RSVQA](#), [VRSBench](#), and [CDVQA](#).
 - **ISRO Sensor Physics**: Formalizes technical specifications for **Cartosat-2S** (0.65m optical pan-sharpened) and **RISAT-1A / EOS-04** (C-band SAR backscatter calibration equations, polarizations VV/VH).
 - **Automated Benchmark Validation**: Writes validation scripts measuring mIoU for grounding, BLEU/CIDEr for VQA, and F1-score for change detection.
 - **Quality Assurance (QA) Testing**: Executes edge-case stress tests (corrupt GeoTIFF headers, unaligned rasters, extreme cloud cover).
 - **Pitch Deck & Presentation Leadership**: Authors the official 8-slide AICTE PPT deck and leads the pitch delivery and jury defense.
 - **Direct Code Ownership**:
 - [satquery/benchmarks/eval_harness.py](#)
 - [satquery/tests/](#) (QA test suites).
 - Curated test dataset manifest ([data/manifest.yaml](#)).
 - Official 8-Slide AICTE Presentation Deck & Jury Defense Battle Card.
 - **Primary Tech Stack**: Python, Scikit-Learn, Torchmetrics, PyTest, Jupyter, PowerPoint.
-

Achintya — Backend Lead & Core Systems Architect

- **Domain Focus**: REST APIs, Schema Contracts, Observability, Telemetry Tracing, System Services.
- **Core Responsibilities**:

- **FastAPI Gateway:** Develops robust REST endpoints (`/api/upload` , `/api/query` , `/api/trace/{id}` , `/api/export-report`).
 - **Pydantic Data Schemas:** Defines universal data contracts (`InputImageMetadata` , `ToolExecutionStep` , `VectorFeature` , `AuditableExecutionTrace` , `QueryResponse`).
 - **Auditable JSON Trace Logger:** Implements real-time telemetry logging recording trace IDs, tool invocations, parameters, latency, and confidence.
 - **Intelligence Dossier Generator:** Develops automated PDF report exporter using `ReportLab` , compiling metadata, preview images, and calculated area metrics.
 - **Backend System Integration:** Connects Misha's geospatial database hooks and Peter's orchestrator into unified request pipelines.
 - **Direct Code Ownership:**
 - `satquery/api/main.py`
 - `satquery/core/schemas.py`
 - `satquery/services/report_generator.py`
 - **Primary Tech Stack:** Python 3.11, FastAPI, Pydantic v2, Uvicorn, ReportLab.
-

Vinayak — Frontend & Web-GIS Lead (with Secondary Backend)

- **Domain Focus:** Geospatial User Interfaces, WebGL Rendering, Interactive Comparison, Secondary REST Integration.
 - **Core Responsibilities:**
 - **Web-GIS Map Canvas:** Builds high-performance interactive map interface using **React** and **MapLibre GL**.
 - **Dual-Pane Split Swipe Slider:** Implements side-by-side interactive swipe tool for bi-temporal (\$T_1\$ vs \$T_2\$) and cross-modal (Optical vs SAR) comparisons.
 - **Vector Layer Overlays:** Renders dynamic GeoJSON bounding boxes and polygon masks with hover tooltips displaying surface area metrics (ha / km²).
 - **Conversational AI Drawer:** Builds sleek chat drawer with operational prompt suggestions ("*Detect flood inundation*", "*Quantify urban growth*").
 - **Auditable Trace Inspector:** Implements real-time telemetry side modal displaying live JSON execution traces for jury inspection.
 - **Secondary Backend Support:** Collaborates with Achintya on backend route integration and file upload streaming.
 - **Direct Code Ownership:**
 - `satquery-frontend/` (Full React / Vite / TypeScript codebase).
 - Secondary helper routes in `satquery/api/routes_frontend.py` .
 - **Primary Tech Stack:** React 18, TypeScript, MapLibre GL / OpenLayers, Tailwind CSS, Lucide Icons, Vite.
-

Misha — Database & Geospatial Pipeline Lead

- **Domain Focus:** Spatial Databases, GeoTIFF Ingestion, Affine Projections, Band Mathematics.
- **Core Responsibilities:**
- **Geospatial Data Lake:** Sets up **PostgreSQL + PostGIS** (or SpatiaLite) for spatial indexing, raster metadata, and vector polygon storage.
- **GeoTIFF Ingestion Pipeline (`GeospatialEngine`):** Parses Cloud-Optimized GeoTIFFs (COG) via `Rasterio` and `GDAL` ; validates CRS (`EPSG:4326` , `EPSG:32643`) and Ground Sample Distance (GSD).

- **Affine Coordinate Transformation:** Implements exact affine projection converting raw pixel masks (x, y) into world coordinates (lon, lat) , mathematically eliminating coordinate hallucinations.
- **Vector Mask Generation:** Converts raster masks into simplified GeoJSON polygons with physical area calculations ([shapely](#)).
- **Spectral & SAR Math:** Implements NDVI, NDWI band slicing, SAR radiometric calibration (σ^0 dB), and Lee speckle filtering (5×5 kernel).
- **Direct Code Ownership:**
- [satquery/services/geospatial.py](#)
- [satquery/db/](#) (PostGIS schemas and spatial queries).
- **Primary Tech Stack:** Rasterio, GDAL, Shapely, PostgreSQL, PostGIS, GeoAlchemy2, NumPy, SciPy.

3. GitHub Repository Structure & Module Ownership

To prevent merge conflicts and ensure complete modularity, each member owns dedicated subdirectories:

```

satquery/
├── api/                                [Achintya + Vinayak]
│   ├── main.py                        # FastAPI server & route definitions
│   └── routes_frontend.py             # Upload streaming & UI helper endpoints
├── benchmarks/                        [Pradipti]
│   ├── eval_harness.py               # RSVQA, VRSBench, CDVQA benchmark runners
│   └── test_splits/                  # Curated benchmark validation subsets
├── core/                              [Peter + Achintya]
│   ├── config.py                     # Constants, dataset paths, parameter bounds
│   └── schemas.py                    # Pydantic data contracts (Inputs, Outputs, Trace)
├── db/                                [Misha]
│   ├── models.py                     # PostGIS spatial tables & raster records
│   └── connection.py                 # Database pool & spatial queries
├── mlops/                             [Chhavi]
│   ├── quantize.py                   # ONNX Runtime FP16 / INT8 export scripts
│   └── gpu_manager.py                 # Dynamic CUDA memory allocation & cleanup
├── services/
│   ├── orchestrator.py               [Peter] # Central Agentic Task Router & Dispatcher
│   ├── geospatial.py                [Misha] # Rasterio GeoTIFF parser & Affine transform
│   └── report_generator.py            [Achintya] # Automated ReportLab PDF Dossier export
├── tests/
│   ├── test_pipeline_smoke.py         # End-to-end integration tests
│   └── test_geospatial_crs.py        # CRS & Affine coordinate validation tests
├── tools/                             [Chhavi]
│   ├── base.py                       # Specialist Tool Base Class with telemetry
│   ├── vqa_engine.py                 # Tool 1: Single VQA & Scene Captioning
│   ├── spatial_grounding.py          # Tool 2: Grounding DINO + SAM-2
│   ├── change_engine.py              # Tool 3: ChangeFormer-V2 CDVQA
│   └── fusion_engine.py               # Tool 4: Optical-SAR Cross-Modal Fusion
└── satquery-frontend/                 [Vinayak]
    ├── src/
    │   ├── components/map/           # MapLibre GL interactive canvas
    │   ├── components/swipe/         # Dual-pane split swipe comparison slider
    │   ├── components/chat/          # Conversational AI drawer & prompts
    │   └── components/trace/         # Live Auditable JSON Trace inspector
    ├── package.json
    └── tailwind.config.js

```

4. The "Zero-Blocker" Mock Contract Strategy

A major risk in hackathons is members waiting on each other. We eliminate this by defining **Mock Contracts on Day 1**:

Zero-Blocker Development Contracts		
Track Interlock	Production Flow	Day 1 Mock Contract (Unblocked)
Misha -> Chhavi (DB -> AI)	Real GeoTIFF parsed arrays and SAR dB.	Misha provides a synthetic 4- band NumPy array (512x512) so Chhavi tests models immediately
Chhavi -> Peter (AI -> Orch.)	Full PyTorch / SAM-2 inference pipeline.	Chhavi provides a mock Python function returning dummy mask & text so Peter wires routing.
Achintya -> Vinayak (BE -> FE)	Live FastAPI backend running on local server.	Achintya provides static JSON mock response file so Vinayak builds Web-GIS UI immediately.
Pradipti -> All (Research -> All)	Full benchmark validation runs.	Pradipti provides 5 curated sample test pairs for immediate feature testing on Day 1.

The Golden JSON API Handshake Contract (Achintya & Peter):

Every tool and endpoint strictly adheres to this Pydantic schema:

```
{
  "trace_id": "satquery-exec-9482",
  "timestamp": "2026-09-04T16:15:00Z",
  "query": "Use the optical and SAR images together to identify built-up and water-covered regions.",
  "task_category": "CROSS_MODAL_JOINT_ANALYSIS",
  "text_response": "Identified 4,820 hectares of urban structures and 1,230 hectares of water bodies using",
  "vector_layers": [
    {
      "layer_name": "cross_modal_water_bodies",
      "feature_count": 18,
      "geojson": { "type": "FeatureCollection", "features": [] },
      "metrics": { "total_area_hectares": 1230.5, "pixel_resolution_m": 10.0 }
    }
  ],
  "confidence_score": 0.95,
  "execution_trace": {
    "trace_id": "satquery-exec-9482",
    "orchestration": { "selected_task": "CROSS_MODAL_JOINT_ANALYSIS", "pipeline_steps": [] }
  }
}
```

5. Git Branching Strategy & Definition of Done

Branching Rules:

- `main` : Protected. Only Peter can merge into `main` after running the smoke test suite.
- `develop` : Integration branch. All features merge here first.
- Feature branches:
 - `feature/peter-orchestrator`
 - `feature/chhavi-ai-models`
 - `feature/pradipti-benchmarks-qa`
 - `feature/achintya-backend-api`
 - `feature/vinayak-frontend-webgis`
 - `feature/misha-geospatial-db`

Definition of Done (DoD) for Any Pull Request:

1. Code adheres strictly to Pydantic schemas in `core/schemas.py`.
 2. Python functions contain docstrings and explicit type hints.
 3. No hardcoded absolute file paths (use `core/config.py`).
 4. PR passes local smoke test: `pytest tests/test_pipeline_smoke.py`.
 5. Peter reviews and merges with zero conflicts.
-

6. 36-Hour Hackathon Hour-by-Hour Battle Plan

Hours 00-06: Setup, Mock Wiring & Baseline Sanity

- └ Peter: Repo init, branching setup, orchestrator skeleton.
- └ Achintya & Vinayak: FastAPI skeleton + React MapLibre canvas init.
- └ Misha: Rasterio GeoTIFF parser, CRS validator, Affine transform test.
- └ Chhavi: Setup PyTorch/CUDA, load SAM-2 and RS-VLM model weights.
- └ Pradipti: Load curated test pairs; verify ground-truth masks.

Hours 06-14: Core Engine Execution & Geospatial Pipeline

- └ Chhavi: Wire RS-VQA captioning, SAM-2 grounding, and ChangeFormer.
- └ Misha: Implement SAR dB calibration, Lee speckle filter, NDVI/NDWI.
- └ Vinayak: Build split-screen swipe slider and vector polygon map layer.
- └ Achintya: Connect specialist tool outputs to Pydantic response models.
- └ Pradipti: Benchmark model accuracy against RSVQA test split.
- └ Peter: Connect intent router with dynamic tool execution pipeline.

Hours 14-22: System Integration & Trace Telemetry

- └ Peter: Implement permitted parameter bounding and JSON trace generator.
- └ Misha & Vinayak: Connect raster-to-GeoJSON polygons with hover tooltips.
- └ Achintya: Build PDF Intelligence Dossier export using ReportLab.
- └ Chhavi: Finalize Optical-SAR cross-modal cross-attention fusion.
- └ Pradipti: Run edge-case stress tests (corrupt headers, cloud simulation).
- └ Vinayak: Connect chat drawer to backend /api/query endpoint.

Hours 22-28: UI/UX Polish, Split Swipe & Trace Inspector

- └ Vinayak: Perfect swipe comparison slider (T1/T2 & Optical/SAR).
- └ Achintya & Vinayak: Build live JSON trace inspector modal in UI.
- └ Chhavi: Quantize models using ONNX Runtime; optimize GPU VRAM memory.
- └ Misha: Simplify polygon boundaries for instant WebGL map rendering.
- └ Pradipti: Review technical Whitepaper; draft jury presentation slides.
- └ Peter: Conduct full-system integration audit; resolve latency bottlenecks.

Hours 28-32: Stress Testing, Offline Fallback & Pitch Deck

- └ Chhavi & Peter: Package offline local tile cache for zero-downtime demo.
- └ Pradipti: Finalize official 8-slide AICTE PPT deck with architecture graphics.
- └ Achintya & Misha: Test custom GeoTIFF upload and report generation.
- └ Vinayak: Final UI styling touch-ups, responsive design, dark mode.
- └ All: Run end-to-end rehearsal with internet disconnected.

Hours 32-36: Live Rehearsals, Defense Prep & Finale

- └ Pradipti & Peter: Run 3 dry-run presentations with strict 8-minute timers.
- └ All Members: Review Jury Q&A Battle Card for specific domain questions.
- └ Peter: Freeze codebase on main branch; create backup demo video recording.
- └ All: Execute winning presentation before the ISRO Evaluation Panel!

7. Jury Q&A Battle Card by Member

- **Peter (Team Leader & Orchestrator):**

- *Jury:* "What makes your orchestrator 'Agentic' instead of just a basic switch statement?"
- *Answer:* "The orchestrator performs dynamic intent reasoning, checks multi-modal metadata compatibility across sensors, selects and sequences tools from our registry, enforces mathematical parameter safety bounds, and synthesizes an auditable JSON execution trace with confidence scoring."

- **Chhavi (AI & MLOps Lead):**

- *Jury:* "How can you execute multiple large models without GPU Out-Of-Memory crashes?"

- *Answer:* "We employ ONNX Runtime FP16 quantization and dynamic CUDA memory management, isolating inference contexts and offloading weights when idle to ensure a sub-2.5 second response time."
- **Pradipti (Research, QA & Pitch Lead):**
- *Jury:* "How do you prove that your solution is domain-adapted to remote sensing rather than generic CV?"
- *Answer:* "We fine-tuned on BigEarthNet-MM (Sentinel-1 SAR and Sentinel-2 optical pairs), benchmarked on RSVQA, VRSBench, and CDVQA, and natively account for sensor physics like SAR backscatter calibration and GSD."
- **Achintya (Backend Lead):**
- *Jury:* "How does the system ensure transparency for defense and space applications?"
- *Answer:* "Every query generates a standardized, verifiable JSON execution trace documenting the trace ID, invoked tools, parameter bounds, confidence scores, and latency, ensuring 100% auditability without black-box opacity."
- **Vinayak (Frontend Lead):**
- *Jury:* "How does your interface help a non-expert disaster relief director?"
- *Answer:* "Our dual-pane split comparison slider enables intuitive side-by-side swipe analysis between dates or optical and SAR captures, with real-time vector overlays detailing exact damage and inundation in hectares."
- **Misha (Database & Geospatial Lead):**
- *Jury:* "How do you prevent the language model from hallucinating geographical coordinates?"
- *Answer:* "The language model never invents coordinates. All bounding polygons are calculated mathematically by projecting pixel mask coordinates through the GeoTIFF's embedded Affine Transformation Matrix into EPSG:4326."

Operational Handbook verified and approved for Peter, Chhavi, Pradipti, Achintya, Vinayak, and Misha — SIH 2026.